第 22 章：用 TaskGroup 管住一组并发任务

一句话先记住：`asyncio.TaskGroup` 不只是“同时启动几个协程”，它还把这些任务的创建、等待、失败传播和清理限制在一个明确的代码作用域里。

这类能力常见于一次请求需要并发调用多个下游的场景。例如，一个 Agent 同时搜索知识库、查询客户记录和获取服务状态。并发本身并不难，真正的工程问题是：一个调用失败后，其余调用由谁停止；用户取消请求后，连接由谁关闭；父请求返回前，怎样保证没有遗漏仍在后台运行的任务。

学完本章，你应该能够：

- 用任务树说明父任务、TaskGroup 和子任务之间的所有权；
- 在 TaskGroup 成功退出后安全收集每个任务的结果；
- 解释一个子任务失败后，兄弟任务取消、资源清理和异常抛出的先后关系；
- 用 `ExceptionGroup` 和 `except*` 处理多个并发错误；
- 区分子任务失败、单个子任务取消、父请求取消和整体超时；
- 在“全部成功才有意义”和“允许部分业务失败”之间选择合适策略。

本章示例只使用 Python 标准库。`TaskGroup`、`ExceptionGroup` 和 `except*` 从 Python 3.11 开始提供；仓库要求 Python 3.14，完整示例也使用了 Python 3.14 支持的 `eager_start` 参数。

22.1 为什么裸 `create_task()` 容易留下生命周期漏洞

下面的代码确实启动了后台任务，但父请求立即返回了：

```
async def handle_request() -> str:
    asyncio.create_task(call_tool())
    return "请求已结束"
```

问题不是 `call_tool()` 能不能运行，而是代码没有回答三个问题：

- 谁保存这个 Task 的引用并等待它结束？
- 父请求被取消时，谁取消这个 Task？
- Task 稍后失败时，谁接收它的异常？

如果没有明确答案，这个 Task 就可能比创建它的请求活得更久，它可能继续占用连接，也可能产生

如果没有明确答案，这个 Task 就可能比创建它的请求活得更久。它可能继续占用连接，也可能产生无人接收的异常。

TaskGroup 把任务组织成一棵有边界的树：

```
handle_request: 承载本次请求的父 Task
└── async with asyncio.TaskGroup() as group
    ├── group.create_task(search_docs())
    ├── group.create_task(load_customer())
    └── group.create_task(fetch_status())
```

父 Task 负责进入和退出 TaskGroup。只有通过 `group.create_task()` 登记的 Task 才是组成员。父 Task 成功越过 `async with` 时，所有组成员都已经结束。

因此，结构化并发解决的是**所有权、生命周期和错误传播**。它不会创建新线程，不保证任务完成顺序，也不限制同时运行的任务数量。

22.2 Java 工程师可以怎样建立心智模型

可以先把 TaskGroup 类比为 Java 的结构化并发作用域，但不能完全等同。两者都希望让子任务的生命周期服从词法作用域；具体调度模型、失败策略和 API 并不相同。

Java 心智模型	Python 对应概念	不能直接等同的地方
结构化并发作用域	`async with TaskGroup()`	TaskGroup 对非取消异常采用固定的兄弟取消与异常聚合规则
在作用域中派生子任务	`group.create_task(coro)`	创建的是事件循环中的 Task， 不是平台线程或虚拟线程
`Future<T>` / `CompletableFuture<T>`	`asyncio.Task[T]`	`task.result()` 不会阻塞等待；未完成时会抛 `InvalidStateError`
`join` 后读取结果	成功退出 TaskGroup 后读取各 Task 结果	等待动作由 `async with` 的退出协议完成， 没有单独的 `join()`
多个失败原因	`ExceptionGroup`	Python 保留一棵异常树，并用 `except*` 按叶子类型切分
`try-finally` / `try-`	协程中的 `try-finally` /	取消通过 `CancelledError` 在可取消点进入协程，

EADME × | readme × | README × | README × | README × | README × | README × | README × | README × | README
'2502x6dg8fpjj0sff40000gp/T/crossnote-f-a066c43932c79fc0393cccf2118.html
eclp相关 | 配运运单26年需求... | 运单小秘值班安排 | 研发日常任务 | 运单运维 | 运单-资源大屏 | 20260618-应急预案 | 运单26年需求迭代... | 商家下单计数

多个失败原因	`ExceptionGroup`	Python 保留一棵异常树，并用 `except*` 按叶子类型切分
`try-finally / try-with-resources`	协程中的 `try-finally / async with`	取消通过 `CancelledError` 在可取消点进入协程，不是硬中断线程

`CompletableFuture.allOf(...)` 也能组合多个异步结果，但它本身不表达"一个请求拥有这些任务，离开作用域前必须完成统一取消和清理"。TaskGroup 的重点不是少写几个 API 调用，而是让任务树直接体现在代码结构中。

还要注意，常见的 asyncio 程序在一个事件循环线程中协作调度。协程执行到 `await` 才把控制权交还事件循环。它适合并发等待 I/O；CPU 密集型同步代码如果长时间不让出控制权，仍会阻塞整个事件循环。

22.3 正常路径：创建任务，退出作用域，再取结果

下面是一个可以独立运行的最小示例：

```
import asyncio


async def call_tool(name: str) -> str:
    await asyncio.sleep(0.1)
    return f"{name}: ok"


async def handle_request() -> list[str]:
    async with asyncio.TaskGroup() as group:
        search = group.create_task(call_tool("search"))
        status = group.create_task(call_tool("status"))

    return [search.result(), status.result()]


print(asyncio.run(handle_request()))
```

执行过程可以拆成四步：

1. 调用 `call_tool(...)` 得到协程对象。调用 `async def` 不会立即执行函数体。
2. `group.create_task(...)` 把协程包装成 Task，登记到组内，并交给事件循环调度。

执行过程可以拆成四步：

1. 调用 `call_tool(...)` 得到协程对象。调用 `async def` 不会立即执行函数体。
2. `group.create_task(...)` 把协程包装成 Task，登记到组内，并交给事件循环调度。
3. 到达 `async with` 末尾时，TaskGroup 的退出协议等待所有成员结束。
4. 成功退出后，两个 Task 都已完成，`result()` 可以直接返回各自结果。

这里保存的 `search` 和 `status` 是 `Task[str]`，不是最终字符串。Python 的 `with` 块不会像 Java 的花括号那样创建局部变量作用域，所以退出块后仍能访问这两个名字。

不要在任务尚未完成时调用 `task.result()`。这个方法不会像 Java 的 `Future.get()` 那样阻塞，而是立即抛出 `asyncio.InvalidStateError`。如果组内逻辑确实依赖某个结果，可以写 `value = await task`；但这样会引入依赖顺序，可能把原本可并发的工作变成分阶段执行。

“退出后可以取结果”还隐含了一个前提：TaskGroup 是**成功退出**的。如果组内有任务失败，`async with` 会直接抛异常组，后面的结果收集语句不会执行。失败任务保留原异常，被取消的任务则处于 cancelled 状态，对它们调用 `result()` 都不会得到正常值。

22.4 一个任务失败后，TaskGroup 会做什么

只要有成员抛出 `CancelledError` 之外的异常，TaskGroup 就开始 fail-fast 关闭。假设 `load_customer` 首先失败，因果顺序是：

```
load_customer 抛出非取消异常
        |
        ▼
TaskGroup 进入关闭状态
        |
        ├── 向仍未完成的 search_docs 请求取消
        ├── 向仍未完成的 fetch_status 请求取消
        └── 若父 Task 仍在组内 await，则用内部取消唤醒它
                    |
                    ▼
子任务在可取消点收到 CancelledError，并执行 finally
                    |
                    ▼
TaskGroup 等到所有成员真正结束
                    |
                    ▼
把非取消异常组成 ExceptionGroup，再抛到作用域外
```

最重要的边界是：**先等所有成员清理结束，再把异常交给组外代码。** 兄弟任务具体按什么顺序收到取消、按什么顺序完成清理，都由调度决定，不应成为业务逻辑的依赖。

取消也不是强制终止。`task.cancel()` 只是提出取消请求。任务通常在下一次 `await` 时收到 `CancelledError`。如果协程正在执行长时间不让出控制权的同步代码，取消就不能及时生效。

资源清理应放在 `finally` 中：

```
async def read_remote(client) -> bytes:
    stream = await client.open_stream()
    try:
        return await stream.read()
    finally:
        await stream.aclose()
```

成功、普通异常和取消都会经过 `finally`。如果确实要捕获 `CancelledError` 来记录日志或补充清理，完成后通常要再次 `raise`，否则 TaskGroup、timeout 以及调用方可能误以为任务正常结束。

如果异常来自 `async with` 块本身，而不是某个子任务，TaskGroup 也会取消并等待所有成员。这个块异常会和子任务产生的非取消异常一起参与聚合。`KeyboardInterrupt` 和 `SystemExit` 是特殊情况：组仍会先取消并等待其他成员，但最终重新抛出原异常，而不是把它当作普通失败包装。

22.5 为什么并发错误需要 ExceptionGroup

第一个失败触发取消，不代表最终只有一个错误。另一个任务可能已经同时失败，也可能在关闭资源时又遇到错误。为了不丢失这些信息，TaskGroup 会用 `ExceptionGroup` 保存异常树：

```
ExceptionGroup: unhandled errors in a TaskGroup
├── RecordNotFound: 客户记录不存在
├── OSError: 关闭响应流失败
└── ValueError: 响应结构不符合约定
```

普通 `except ExceptionGroup` 抓到的是整个容器。`except* SomeError` 则递归匹配异常树中的叶子，把匹配部分保留为一个子组交给当前处理器，未匹配部分继续交给后续处理器或向外传播。

```
try:
    async with asyncio.TaskGroup() as group:
        group.create_task(search_docs())
```

:ADME × | readme × | README × | README × | README × | README × | README × | README × | README × | README
2502x6dg8fpjj0sff40000gp/T/crossnote-f-a066c43932c79fc0393cccf2118.html
eclp相关 | 配运运单26年需求... | 运单小秘值班安排 | 研发日常任务 | 运单运维 | 运单-资源大屏 | 20260618-应急预案 | 运单26年需求迭代... | 商家下单计数

```
try:
    async with asyncio.TaskGroup() as group:
        group.create_task(search_docs())
        group.create_task(load_customer())
        group.create_task(fetch_status())
except* RecordNotFound as expected_group:
    log_expected_failures(expected_group)
except* OSError as io_group:
    log_resource_failures(io_group)
```

`expected_group` 和 `io_group` 都是异常组，不是单个异常。即使只匹配到一个叶子，处理器拿到的仍然是组。这样可以保留原异常树的层次和 traceback。

使用 `except*` 时还要记住这些语法边界：

- 同一个 `try` 只能使用 `except` 或 `except*`，不能混用；需要两种处理时嵌套两层 `try`；
- `except*` 后必须写异常类型，不存在裸 `except*:`；
- `return`、`break` 和 `continue` 不能出现在 `except*` 子句中；
- 没有被任何 `except*` 处理的部分不会消失，而会保持异常组结构继续传播。

不要为了遍历方便就把异常组压平。嵌套 TaskGroup 会自然形成嵌套异常组，保留结构有助于判断故障来自哪一层并发作用域。

22.6 不要混淆失败、取消和超时

这四种退出原因会经过相似的清理路径，但最终语义不同：

情况	TaskGroup 怎样处理兄弟任务	组外通常看到什么
一个成员抛非取消异常	取消未完成兄弟并等待清理	`ExceptionGroup` 或 `BaseExceptionGroup`
一个成员被取消或自行取消	不因为这一项取消而自动取消其他成员	若无其他失败，组可以正常退出
承载 TaskGroup 的父 Task 被取消	取消未完成成员并等待清理	原来的 `CancelledError` 继续传播
外层 `asyncio.timeout()` 到期	先取消父 Task，再由组取消并等待成员	timeout 作用域外收到 `TimeoutError`

单个成员被取消并不等于它返回了一个空值。这个 Task 的 `cancelled()` 为 `True`，调用

单个成员被取消并不等于它返回了一个空值。这个 Task 的 `cancelled()` 为 `True`，调用 `result()` 会重新抛出 `CancelledError`；只是 TaskGroup 不把这一次取消当作触发整组失败的普通异常。

`asyncio.CancelledError` 直接继承 `BaseException`，而不是 `Exception`。这能降低它被普通业务异常处理误吞的概率，但不代表可以随意写 `except BaseException`。下面的包装会把用户停止请求也伪装成一次普通失败：

```
async def wrong(operation):
    try:
        return await operation
    except BaseException as error:
        return {"ok": False, "error": str(error)}
```

正确原则是：取消属于控制流，清理后继续传播；只有业务已经定义为“允许恢复”的异常，才转换成普通结果。

22.7 两种常见业务策略

TaskGroup 提供统一的生命周期规则，但不会替你决定业务上允许失败到什么程度。关键问题是：缺少一个子任务结果时，本次请求是否仍然有意义且安全？

全部必需：让异常自然离开子任务

如果生成合规答复必须同时拿到政策、客户记录和实时状态，缺一项就应终止请求：

```
async with asyncio.TaskGroup() as group:
    policy = group.create_task(load_policy())
    customer = group.create_task(load_customer())
    status = group.create_task(fetch_status())
```

不要在每个工具调用中用宽泛的 `except Exception` 把错误变成 `None`。那会使 TaskGroup 误以为任务全部正常完成，也会让调用方拿着不完整数据继续执行。

允许部分业务失败：只转换明确可恢复的异常

如果“没有找到客户记录”本来就是可展示的业务结果，可以在子任务内部把这一种异常转换成显式数据：

```python
from collections.abc import Awaitable, Callable
from dataclasses import dataclass


class RecordNotFound(Exception):
    pass


@dataclass(frozen=True)
class ToolResult:
    name: str
    value: str | None,
    error: str | None


async def capture_business_failure(
    name: str,
    operation: Callable[[], Awaitable[str]],
) -> ToolResult:
    try:
        return ToolResult(name, await operation(), None)
    except RecordNotFound as error:
        return ToolResult(name, None, str(error))
```

使用时，每个包装器本身成为 TaskGroup 的成员：

```python
async with asyncio.TaskGroup() as group:
    search = group.create_task(capture_business_failure("search", search_docs))
    customer = group.create_task(capture_business_failure("customer", load_customer))

results = [search.result(), customer.result()]
```

这里故意传入函数，而不是提前调用函数得到协程。包装器开始执行后才创建并等待真正的协程，任务的创建与消费边界更清楚。

这个包装器不会捕获 `TypeError` 、协议解析错误等未知缺陷，也不会吞掉 `CancelledError` 。这些情况仍会触发 TaskGroup 的正常失败或取消流程。“best effort”不等于“忽略所有异常”。

可以用下面的决策表选择策略：

README × | README × | README × | README × | README × | README × | README × | README × | README × | README
2502x6dg8fpjj0sff40000gp/T/crossnote-f-a066c43932c79fc0393cccf2118.html
eclp相关 | 配运运单26年需求... | 运单小秘值班安排 | 研发日常任务 | 运单运维 | 运单-资源大屏 | 20260618-应急预案 | 运单26年需求迭代... | 商家下单计数

业务含义	处理方式
少一项就不能正确回答或提交	让异常离开子任务，由 TaskGroup 终止整组
某种已知业务失败也是合法结果	只捕获该具体异常，转换成结构化结果
未知程序、协议或数据不变量错误	不降级为部分成功，继续传播
用户取消或上游断开	清理后继续传播 `CancelledError`

22.8 嵌套调用不会自动继承任务所有权

TaskGroup 不会扫描调用栈，也不会收编当前上下文中所有由 `asyncio.create_task()` 创建的任务：

```
async def launch_background() -> None:
    asyncio.create_task(background_work())  # 不属于调用方的 TaskGroup


async with asyncio.TaskGroup() as group:
    group.create_task(launch_background())
```

组只拥有 `launch_background()` 对应的 Task。这个 Task 返回后， `background_work()` 仍可能继续运行。正确设计通常有两种。

第一种是把现有组显式传给下层，让新任务属于同一个请求：

```
def add_request_tasks(group: asyncio.TaskGroup) -> None:
    group.create_task(background_work())
```

第二种是让下层建立自己的嵌套作用域，并且退出自己的组以后再返回：

```
async def run_nested_work() -> None:
    async with asyncio.TaskGroup() as nested_group:
        nested_group.create_task(background_work())
```

可以把 TaskGroup 作为参数，但不要把它藏在全局变量里。显式参数让函数签名直接说明新任务属于哪个生命周期。如果某项后台工作本来就应该跨越请求长期运行，它应当归属于应用级服务，并有独立的启动、停止和异常处理协议。

ADME × | readme × | README × | README × | README × | README × | README × | README × | README × | README ×
!502x6dg8fpjj0sff40000gp/T/crossnote-f-a066c43932c79fc0393cccf2118.html
eclp相关 | 配运运单26年需求... | 运单小秘值班安排 | 研发日常任务 | 运单运维 | 运单-资源大屏 | 20260618-应急预案 | 运单26年需求迭代... | 商家下单计数

22.9 整体超时应该包住整个任务组

如果几个调用共享同一份请求预算，把 `asyncio.timeout()` 放在 TaskGroup 外层：

```
try:
    async with asyncio.timeout(2.0):
        async with asyncio.TaskGroup() as group:
            group.create_task(search_docs())
            group.create_task(load_customer())
            group.create_task(fetch_status())
except TimeoutError:
    handle_timeout()
```

到期后的数据流是：

1. timeout 取消当前父 Task；
2. 父 Task 退出 TaskGroup，组取消仍未完成的成员；
3. TaskGroup 等待子任务执行清理；
4. 取消回到 timeout 作用域；
5. timeout 把自己触发的取消转换成 `TimeoutError`，在作用域外抛出。

因此，两秒预算不等于函数一定在墙钟时间 2.000 秒时强制返回。事件循环要有机会投递取消，资源清理也可能需要时间。超时能约束等待协议，但不能撤销远端已经产生的副作用。

`TimeoutError` 必须在 timeout 上下文外捕获，因为异常转换发生在退出上下文时。普通的上游取消也不会被随意转换成超时；timeout 只转换由它自己触发的取消。

22.10 TaskGroup 与 `gather()` 怎样选择

`asyncio.gather()` 仍适用于需要按输入顺序聚合结果的代码，也常见于旧项目。但它的默认失败行为与 TaskGroup 不同：

维度	`TaskGroup`	`gather(..., return_exceptions=False)`
代码边界	`async with` 明确表示任务作用域	调用方等待一个聚合 awaitable
一个成员普通失败	取消未完成兄弟，等其清理后抛异常组	立即向等待者传播该异常，其他成员默认继续运行
多个失败	关闭完成后聚合为异常组	默认传播首先观察到的异常

README × | readme × | README × | README × | README × | README × | README × | README × | README × | README
z72502x6dg8fpjj0sff40000gp/T/crossnote-f-a066c43932c79fc0393cccf2118.html
eclp相关 | 配运运单26年需求... | 运单小秘值班安排 | 研发日常任务 | 运单运维 | 运单-资源大屏 | 20260618-应急预案 | 运单26年需求迭代... | 商家下单计数

结果收集	保存 Task，成功退出组后读取	返回列表，顺序与输入一致
适用重点	请求内的任务所有权、失败联动与清理	结果排序或必须兼容的既有聚合语义

`gather()` 自身被取消时，会把取消传播给尚未完成的成员；但这与"某个成员普通失败"不是一回事。
使用 `return_exceptions=True` 可以把异常放进结果列表，却也更容易把程序缺陷与业务失败混在一起。

新写的请求级并发通常优先选择 TaskGroup。只有确实需要 gather 的返回值语义，.并且能明确说明异常后谁继续拥有、等待和清理其他任务时，再使用 gather。

另外，TaskGroup 不负责限流。循环调用一千次 `group.create_task()`，就可能登记一千个任务。
限制并发量要结合 `asyncio.Semaphore`、连接池或速率限制器，这是第 23 章的主题。

22.11 Java 工程师最容易踩的坑

误判或错误写法	实际表现	正确做法
`task.result()` 等同于 `Future.get()`	未完成时立即抛 `InvalidStateError`	`await task`，或成功退出组后读取
TaskGroup 会返回一张结果列表	TaskGroup 管生命周期，不直接聚合结果	保存 `create_task()` 返回的 Task
一个任务失败后所有代码立刻停止	取消需要到达可取消点，清理也需要时间	用 `finally` 清理，并等待作用域退出
捕获 `Exception` 就是 best effort	未知程序缺陷也会被伪装成业务失败	只捕获已定义恢复语义的具体异常
捕获 `BaseException` 更保险	会吞掉取消和进程退出类信号	不宽捕获；捕获取消后通常重新抛出
普通 `except ValueError` 能匹配组内叶子	外层对象是异常组	使用 `except* ValueError`
嵌套函数裸建 Task 会自动入组	新 Task 不属于当前组	传入现有组，或在下层自建嵌套组
TaskGroup 会限制并发数量	它只管理所有权和失败传播	另加 Semaphore、连接池或速率限制
超时会强制终止工作	取消是协作式的，远端副作用也不会回滚	设计幂等、清理和服务端超时

22.12 运行本章示例

本章包含两个文件：

文件	作用
`README.md`	建立 TaskGroup、取消、异常组和失败策略的心智模型
`example.py`	用 `asyncio.Event` 精确控制时序，并用断言验证各条路径

`example.py` 不访问网络，也没有第三方依赖。它依次验证：

- 正常退出后读取结果；
- 一个任务失败时取消兄弟并等待清理；
- 多个失败的聚合与 `except*` 分型；
- 部分业务失败与未知程序缺陷的不同处理；
- 单个成员取消与父请求取消的差别；
- 裸 `create_task()` 产生的游离任务；
- 整体 timeout 的清理顺序；
- gather 传播异常后兄弟任务仍继续运行。

从仓库根目录运行：

```
.venv/bin/python -B java_to_python/chapters/22_asyncio_taskgroup/example.py
```

成功时最后会输出：

```
所有第 22 章示例与运行时断言均已完成。
```

可继续执行静态检查：

```
.venv/bin/ruff check java_to_python/chapters/22_asyncio_taskgroup
.venv/bin/ruff format --check java_to_python/chapters/22_asyncio_taskgroup
.venv/bin/pyright java_to_python/chapters/22_asyncio_taskgroup/example.py
```

这些内存替身能验证 Python 层面的任务、取消、异常和清理语义，但不能代替真实 HTTP、数据库、LLM 或 MCP 客户端的集成测试。真实系统还要验证连接池、驱动取消、服务端断连、重试、幂等性以及远端副作用。

README × readme × README × README × README × README × README × README × README × README
7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-a066c43932c79fc0393cccf2118.html
eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

- 整体 timeout 的清理顺序；
- gather 传播异常后兄弟任务仍继续运行。

从仓库根目录运行：

```
.venv/bin/python -B java_to_python/chapters/22_asyncio_taskgroup/example.py
```

成功时最后会输出：

```
所有第 22 章示例与运行时断言均已完成。
```

可继续执行静态检查：

```
.venv/bin/ruff check java_to_python/chapters/22_asyncio_taskgroup
.venv/bin/ruff format —check java_to_python/chapters/22_asyncio_taskgroup
.venv/bin/pyright java_to_python/chapters/22_asyncio_taskgroup/example.py
```

这些内存替身能验证 Python 层面的任务、取消、异常和清理语义，但不能代替真实 HTTP、数据库、LLM 或 MCP 客户端的集成测试。真实系统还要验证连接池、驱动取消、服务端断连、重试、幂等性以及远端副作用。

Python 3.14 官方文档可作为进一步参考：

- `asyncio.TaskGroup`
- `asyncio.gather`
- `asyncio.timeout`
- `ExceptionGroup`
- `except*`

22.13 本章小结

TaskGroup 最核心的价值，是让父任务在离开作用域前对所有子任务负责。正常路径会等待所有成员；一个成员发生非取消失败时，会取消仍在运行的兄弟，等它们清理后再聚合错误；父请求取消时，同样先清理孩子，再把取消继续交还调用方。

工程上要同时守住两条边界：用 `group.create_task()` 明确任务所有权，只把已知且可恢复的业务异常转换成普通结果。做到这两点，TaskGroup 才不只是并发语法，而是可靠的请求生命周期边界。

```python
"""第 22 章：TaskGroup、结构化并发与异常组。

示例只用 Event 和本地替身控制执行阶段，不访问网络，也不依赖随机延迟。
每一节都用运行时断言验证任务所有权、失败传播和清理边界。
"""

# 建议按文件末尾 run_examples() 的调用顺序阅读。前面的类型和辅助函数负责搭建可控的
# “假工具”，后面的九个实验再分别放行、取消或制造失败，并通过 events 还原因果顺序。
# 全部实验都运行在同一个线程的事件循环里；这里的“并发”是协程在 await 处交还执行权后
# 交替推进，不等于 Java 同时启动了多个工作线程。

# 三个导入都来自标准库：asyncio 提供协程、Task、Event、TaskGroup、timeout 等并发原语；
# Awaitable[str] 描述“可以被 await，完成后得到 str”的对象类型；dataclass 装饰器则会根据
# 字段声明生成 __init__、__repr__、__eq__ 等样板方法。类型提示只供静态检查和阅读，
# Python 运行时不会像 Java 泛型那样自动验证传入对象及其最终结果。
import asyncio
from collections.abc import Awaitable
from dataclasses import dataclass


# 这两个异常类型把“可预期的业务失败”和“代码/协议缺陷”分开。
# 后面的 best-effort 包装器只恢复前者，让后者继续触发 TaskGroup 的失败联动。
class ToolBusinessError(Exception):
    """调用成功到达工具，但得到可预期、可展示给调用方的业务失败。"""


class ToolProgramError(RuntimeError):
    """协议、解析或程序不变量被破坏；不能伪装成一次普通的部分失败。"""


# @dataclass(...) 在类定义完成时改造类，类似替轻量 POJO 生成样板代码，但它不是 Java 注解：
# 默认并不会根据 name: str 等类型提示做运行时校验。slots=True 让实例使用固定槽位保存字段，
# 因而不能临时添加未声明属性；这里也能尽早暴露示例中的字段名拼错。
@dataclass(slots=True)
class ControlledTool:
    """一个可由测试事件精确放行的工具替身。"""
```

```
35  class ControlledTool:
38      # name 用于日志；value 是放行后要返回的固定结果。
39      name: str
40      value: str
41      # 每个工具共享同一 events 列表，因此 append 会汇入同一条时间线，而不是复制列表。
42      events: list[str]
43      # started 由工具 set、父协程 wait，用来确认工具已运行到 I/O 阶段。
44      started: asyncio.Event
45      # release 的方向相反：父协程 set 后，工具的 wait 才能继续。
46      release: asyncio.Event
47      # X | None 是联合类型，表示可保存一个异常实例或 None；默认 None 代表成功路径。
48      error: Exception | None = None
49
50
51  # frozen=True 会阻止字段在构造后被重新赋值，让收集到的结果更接近 Java 的不可变 record；
52  # 但字段引用的对象若本身可变，frozen 并不会递归冻结它。本类字段恰好都是 str 或 None。
53  @dataclass(frozen=True, slots=True)
54  class ToolResult:
55      """best-effort 策略的显式结果；错误和值不会混在同一字符串里。"""
56
57      # 成功时 value 有值且 error 为 None；业务失败时两者相反。显式字段比用特殊字符串
58      # 同时表示“值”和“错误”更容易判断，也避免把空结果误当成功。
59      name: str
60      value: str | None
61      error: str | None
62
63      # @property 让调用方以 result.ok 读取计算结果，看起来像字段；实际每次读取都会调用方法。
64      @property
65      def ok(self) -> bool:
66          # is 比较对象身份；None 是单例哨兵，因此应写 is None，而不是套用 Java 的 == null。
67          return self.error is None
68
69
70  # 普通 def 是同步函数，调用后立即打印，不产生协程对象。
71  def print_section(title: str) -> None:
72      # f-string 会计算花括号中的 title；开头的 \n 让不同实验之间空一行。
```

```
69
70  # 普通 def 是同步函数，调用后立即打印，不产生协程对象。
71  def print_section(title: str) -> None:
72      # f-string 会计算花括号中的 title；开头的 \n 让不同实验之间空一行。
73      print(f"\n=== {title} ===")
74
75
76  def record(events: list[str], message: str) -> None:
77      """事件既打印又保留，断言只检查因果关系，不猜兄弟任务的调度顺序。"""
78      # append 原地修改调用方传入的同一个列表，随后打印同一条消息供人观察。
79      events.append(message)
80      print(f"  事件 | {message}")
81
82
83  def make_three_tools(
84      events: list[str],
85      *,
86      record_error: Exception | None = None,
87  ) -> tuple[ControlledTool, ControlledTool, ControlledTool]:
88      """每次实验创建全新的事件，避免一个实验的状态污染下一个实验。"""
89      # 参数列表中的 * 表示它后面的 record_error 只能按名字传入，例如
90      # make_three_tools(events, record_error=error)，不能把它作为第二个位置参数。
91      # 返回值是固定三个元素的 tuple，调用方随后可解包为 search、record_call、status。
92      return (
93          # 每次 asyncio.Event() 都创建独立事件对象；初始未置位，wait() 会暂停。
94          ControlledTool(
95              "search_docs",
96              "文档：退款期为 7 天",
97              events,
98              asyncio.Event(),
99              asyncio.Event(),
100         ),
101         ControlledTool(
102             "lookup_record",
103             "记录：客户等级 gold",
104             events,
105             asyncio.Event(),
```

```
 83  def make_three_tools(
101          ControlledTool(
102              "lookup_record",
103              "记录：客户等级 gold",
104              events,
105              asyncio.Event(),
106              asyncio.Event(),
107              # 只把可选错误注入 lookup_record，便于实验精确指定唯一的首发失败项。
108              record_error,
109          ),
110          ControlledTool(
111              "fetch_status",
112              "状态：服务正常",
113              events,
114              asyncio.Event(),
115              asyncio.Event(),
116          ),
117      )
118
119
120  async def execute_controlled_tool(tool: ControlledTool) -> str:
121      """模拟"获取资源 -> 等待 I/O -> 返回/失败 -> finally 清理"的完整调用。"""
122      # 调用 async def 只会先得到协程对象；当它被 Task 调度并实际执行到这里时，才记录开始。
123      record(tool.events, f"{tool.name}：开始并获取资源")
124      # set() 不会阻塞，它把事件置为已触发并唤醒等待 started.wait() 的父协程；在调用 clear()
125      # 之前，后续 wait() 也会立即返回。本例从不 clear，因此每个 started 只承担一次就绪信号。
126      tool.started.set()
127
128      try:
129          # release 初始未置位，所以 await 暂停当前 Task，把事件循环执行权交给其他任务。
130          # 这代表可取消的真实 I/O 等待点，而不是阻塞线程的同步等待。
131          await tool.release.wait()
132          if tool.error is not None:
133              # type(error).__name__ 取异常对象所属类的名字，只用于生成稳定的观察日志。
134              record(tool.events, f"{tool.name}：抛出 {type(tool.error).__name__}")
135              raise tool.error
136
```

```
120  async def execute_controlled_tool(tool: ControlledTool) -> str:
136
137          # 只有 release 已放行且没有注入错误才返回固定值；return 会把 str 保存为 Task 的结果。
138          record(tool.events, f"{tool.name}: 返回成功")
139          return tool.value
140      except asyncio.CancelledError:
141          # cancel() 只是发出请求；本任务在上面的 await 等可取消点收到 CancelledError 后到达这里。
142          # 只记录再裸 raise：裸 raise 会重新抛出当前异常，让取消状态继续走出工具调用。
143          # 若像普通 Exception 那样吞掉它，TaskGroup/timeout 就可能误判任务已经正常完成。
144          record(tool.events, f"{tool.name}: 收到取消")
145          raise
146      finally:
147          # finally 在成功、业务/程序异常和取消三条路径上都会执行。真实代码会在这里归还连接、
148          # 关闭流或回滚事务；TaskGroup 必须等这一段结束后才能完成退出。
149          record(tool.events, f"{tool.name}: 资源清理完成")
150
151
152  # 三个名字模拟不同业务工具，但目前都委托给同一个受控执行器。这里保留独立函数，是为了让
153  # Task 的名称、调用关系和真实 Agent 中三个不同客户端调用一致。
154  async def search_docs(tool: ControlledTool) -> str:
155      # await 不只是“取结果”：它会暂停当前包装协程，直到下层协程成功返回或抛出异常。
156      return await execute_controlled_tool(tool)
157
158
159  async def lookup_record(tool: ControlledTool) -> str:
160      return await execute_controlled_tool(tool)
161
162
163  async def fetch_status(tool: ControlledTool) -> str:
164      return await execute_controlled_tool(tool)
165
166
167  async def wait_until_all_started(*tools: ControlledTool) -> None:
168      """等待事实而不是等待一个猜测的秒数。"""
169      # 函数签名里的 *tools 把任意个位置参数收集成 tuple。循环逐个 await 看似串行，但已 set 的
170      # Event.wait() 会立即返回；最终效果是直到每个工具都报告 started 才结束。
171      for tool in tools:
```

```python
166
167 async def wait_until_all_started(*tools: ControlledTool) -> None:
168     """等待事实而不是等待一个猜测的秒数。"""
169     # 函数签名里的 *tools 把任意个位置参数收集成 tuple。循环逐个 await 看似串行，但已 set 的
170     # Event.wait() 会立即返回；最终效果是直到每个工具都报告 started 才结束。
171     for tool in tools:
172         await tool.started.wait()
173
174
175 def release_all(*tools: ControlledTool) -> None:
176     # set() 是同步操作；循环一次性解除所有工具的等待，之后具体谁先完成由事件循环调度决定。
177     for tool in tools:
178         tool.release.set()
179
180
181 def assert_cleaned_before(
182     events: list[str],
183     marker: str,
184     *tool_names: str,
185 ) -> None:
186     # list.index 返回某消息第一次出现的位置；若消息缺失会直接抛 ValueError，使实验失败。
187     marker_index: int = events.index(marker)
188     for name in tool_names:
189         # 这里只要求每个工具的清理都早于父边界，不要求兄弟之间以某个固定顺序清理。
190         assert events.index(f"{name}: 资源清理完成") < marker_index
191
192
193 async def demonstrate_normal_task_group() -> None:
194     # 第一个实验走纯成功路径：三个工具先全部停在 Event.wait()，父协程确认 Task 尚未完成，
195     # 再统一放行。它要证明结果应在成功退出 TaskGroup 后读取。
196     print_section("1. 最小正确用法：退出 TaskGroup 后再读结果")
197     events: list[str] = []
198     # tuple 按位置解包：三个新 ControlledTool 分别绑定到三个局部变量。
199     search: ControlledTool, record_call: ControlledTool, status: Controlle… = make_three_tools(events)
200
201     # async with 会调用 TaskGroup 的异步进入/退出协议。进入后组变为 active；到达块末尾时，
202     # __aexit__ 会隐式等待所有登记任务结束，因此它相当于一个有明确所有权的并发作用域。
```

```python
async def demonstrate_normal_task_group() -> None:

    # async with 会调用 TaskGroup 的异步进入/退出协议。进入后组变为 active；到达块末尾时，
    # __aexit__ 会隐式等待所有登记任务结束，因此它相当于一个有明确所有权的并发作用域。
    async with asyncio.TaskGroup() as group:
        # create_task 接收协程对象，创建并登记 asyncio.Task，返回值不是最终字符串而是 Task 句柄。
        # name 便于调试；Python 3.14 的 eager_start=False 明确让任务先进入常规调度队列，
        # 不在 create_task 调用栈里立刻推进协程。后面的 await 才给事件循环调度它们的机会。
        search_task: Task[str] = group.create_task(
            search_docs(search),
            name="search_docs",
            eager_start=False,
        )
        record_task: Task[str] = group.create_task(
            lookup_record(record_call),
            name="lookup_record",
            eager_start=False,
        )
        status_task: Task[str] = group.create_task(
            fetch_status(status),
            name="fetch_status",
            eager_start=False,
        )

        # 等三个 started 全部置位后，可以确定任务已启动且停在各自的 release.wait()。
        await wait_until_all_started(search, record_call, status)
        print(
            "  组内尚未放行，三个 Task 都未完成:",
            # 生成器表达式逐个调用 done()；any 为 False，再由 not 变为 True，说明无一完成。
            not any(task.done() for task in (search_task, record_task, status_task)),
        )
        # 此处若调用 task.result() 会抛 InvalidStateError，因为 result() 不会像 Java Future.get()
        # 那样阻塞等待。先解除 I/O，再让作用域退出并隐式等待全部任务。
        release_all(search, record_call, status)

    # 能执行到这里，说明 TaskGroup 已成功退出，三个任务及其 finally 都已完成。
    record(events, "父请求：已经退出 TaskGroup")
```

```python
async def demonstrate_normal_task_group() -> None:

    # 能执行到这里，说明 TaskGroup 已成功退出，三个任务及其 finally 都已完成。
    record(events, "父请求：已经退出 TaskGroup")
    # 列表按固定的 Task 引用顺序取结果；完成的 Task.result() 才安全地返回各自字符串。
    results: list[str] = [search_task.result(), record_task.result(), status_task.result()]
    print("  安全读取结果:", results)

    # 前两条断言验证值与 Task 状态，最后一条验证所有资源清理都发生在作用域退出标记之前。
    assert results == [search.value, record_call.value, status.value]
    assert all(task.done() for task in (search_task, record_task, status_task))
    assert_cleaned_before(
        events,
        "父请求：已经退出 TaskGroup",
        search.name,
        record_call.name,
        status.name,
    )


async def demonstrate_all_required_failure() -> None:
    # “全部必需”表示缺少任一工具结果，本次请求就没有继续价值。这里让 lookup_record
    # 发生程序错误，观察 TaskGroup 如何取消两个尚未完成的兄弟并在清理后聚合异常。
    print_section("2. 全部必需：一项失败，取消兄弟并等清理")
    events: list[str] = []
    search: ControlledTool, record_call: ControlledTool, status: Controlle… = make_three_tools(
        events,
        # 只有 record_call 持有错误；search 和 status 不放行，稳定保持在可取消点。
        record_error=ToolProgramError("记录响应无法解析"),
    )
    # except* 收到异常组后会把叶子数量写入这里，供作用域外断言。
    caught_count = 0

    try:
        async with asyncio.TaskGroup() as group:
            # 三个 Task 都登记到同一个 group，因而共享同一失败和清理边界。
            search_task: Task[str] = group.create_task(search_docs(search), name="search_docs")
```

```
265     try:
266         async with asyncio.TaskGroup() as group:
267             # 三个 Task 都登记到同一个 group，因而共享同一失败和清理边界。
268             search_task: Task[str] = group.create_task(search_docs(search), name="search_docs")
269             record_task: Task[str] = group.create_task(lookup_record(record_call), name="lookup_record")
270             status_task: Task[str] = group.create_task(fetch_status(status), name="fetch_status")
271 
272             await wait_until_all_started(search, record_call, status)
273             # 只放行失败项；另外两项确定仍停在可取消的 Event.wait() 中。record_call 恢复后
274             # 抛 ToolProgramError，TaskGroup 开始 fail-fast 关闭并取消 search/status。
275             record_call.release.set()
276             try:
277                 # 新 Event 永远不会自行 set，这个 await 故意让父 Task 留在 with 块里。
278                 await asyncio.Event().wait()
279             finally:
280                 # 子任务失败会用一次内部取消唤醒仍在 with 块中的父 Task，所以父 Task 的
281                 # finally 得以执行。这个内部取消用于驱动 __aexit__，最终不会以 CancelledError
282                 # 穿过 TaskGroup；组外收到的是由成员失败构成的 ExceptionGroup。
283                 record(events, "父请求：with 块中的等待被中断")
284     # except* 不是捕获外层容器后手工遍历；它会从异常树中选出所有匹配叶子，并保持组结构
285     # 绑定给 error_group。没有匹配的叶子才会继续传播给外层。
286     except* ToolProgramError as error_group:
287         caught_count: int = len(error_group.exceptions)
288         record(events, "父请求：收到 ToolProgramError 异常组")
289 
290     # cancelled() 区分因取消结束的兄弟；失败任务不是 cancelled，而是保存了原异常。
291     # exception() 在 Task 已完成后返回其异常对象，本例用 isinstance 验证类型没有被吞掉。
292     print("  失败叶子数:", caught_count)
293     print("  两个兄弟 Task 均已取消:", search_task.cancelled() and status_task.cancelled())
294     assert caught_count == 1
295     assert search_task.cancelled()
296     assert status_task.cancelled()
297     assert isinstance(record_task.exception(), ToolProgramError)
298     assert_cleaned_before(
299         events,
300         "父请求：收到 ToolProgramError 异常组"
```

```
252 async def demonstrate_all_required_failure() -> None:
298     assert_cleaned_before(
299         events,
300         "父请求：收到 ToolProgramError 异常组",
301         search.name,
302         record_call.name,
303         status.name,
304     )
305
306
307 async def primary_group_failure(
308     started: asyncio.Event,
309     release: asyncio.Event,
310     events: list[str],
311 ) -> None:
312     # 这个任务先报告“我已进入可控阶段”，再等待父协程统一触发真正的主失败。
313     record(events, "主失败任务：已就绪")
314     started.set()
315     await release.wait()
316     record(events, "主失败任务：抛出 ToolBusinessError")
317     raise ToolBusinessError("文档索引中没有该条目")
318
319
320 async def fail_while_cancelling(
321     name: str,
322     started: asyncio.Event,
323     events: list[str],
324     error: Exception,
325 ) -> None:
326     """异常组实验：模拟任务收到取消后，自己的清理又发生异常。"""
327     record(events, f"{name}：已进入等待")
328     started.set()
329     try:
330         # 事件对象没有保存在别处，因此不会被 set；正常情况下这个任务会一直等待。
331         await asyncio.Event().wait()
332     except asyncio.CancelledError:
333         # 主任务失败后，TaskGroup 取消本任务，本例故意让取消清理再抛一个指定异常
```

```
async def fail_while_cancelling(
    try:
        # 事件对象没有保存在别处，因此不会被 set；正常情况下这个任务会一直等待。
        await asyncio.Event().wait()
    except asyncio.CancelledError:
        # 主任务失败后，TaskGroup 取消本任务。本例故意让取消清理再抛一个指定异常，
        # 从而确定性地产生多个并发错误；真实代码只应在清理确实失败时出现这种结果。
        record(events, f"{name}: 取消清理时抛出 {type(error).__name__}")
        # raise error from None 抛出传入的异常，并隐藏“处理 CancelledError 时又抛异常”的
        # 隐式上下文显示，使演示聚焦最终异常组；它与裸 raise 的重新抛取消语义不同。
        raise error from None


async def demonstrate_exception_groups() -> None:
    # 本实验稳定制造三个叶子：主任务的 ToolBusinessError，以及两个兄弟响应取消时产生的
    # OSError、ValueError。随后用 except* 分型处理前两类，让第三类继续传播。
    print_section("3. ExceptionGroup: except* 分型，未处理部分继续传播")
    events: list[str] = []
    # 每个 started Event 分别确认一个任务已就绪；release_primary 只放行主失败任务。
    primary_started = asyncio.Event()
    os_cleanup_started = asyncio.Event()
    value_cleanup_started = asyncio.Event()
    release_primary = asyncio.Event()
    # 两个列表收集已被对应 except* 处理的叶子消息。
    handled_business: list[str] = []
    handled_os: list[str] = []
    # 尚未收到剩余子组时用 None 表示；外层 except 命中后再保存 ExceptionGroup。
    remaining_group: ExceptionGroup | None = None

    # Python 禁止在同一 try 上混用 except 与 except*，所以需要两层：内层按叶子类型切分，
    # 外层把切分后仍未匹配的异常组当作一个普通 ExceptionGroup 接住，以便继续观察。
    try:
        try:
            async with asyncio.TaskGroup() as group:
                # Task 句柄在此实验中无需单独读结果；TaskGroup 自己持有成员并负责等待。
                group.create_task(
```

```
341  async def demonstrate_exception_groups() -> None:
359      try:
360          try:
361              async with asyncio.TaskGroup() as group:
362                  # Task 句柄在此实验中无需单独读结果；TaskGroup 自己持有成员并负责等待。
363                  group.create_task(
364                      primary_group_failure(primary_started, release_primary, events),
365                      name="primary-failure",
366                  )
367                  group.create_task(
368                      fail_while_cancelling(
369                          "日志清理任务",
370                          os_cleanup_started,
371                          events,
372                          OSError("写入清理日志失败"),
373                      ),
374                      name="os-cleanup-failure",
375                  )
376                  group.create_task(
377                      fail_while_cancelling(
378                          "状态清理任务",
379                          value_cleanup_started,
380                          events,
381                          ValueError("清理状态格式错误"),
382                      ),
383                      name="value-cleanup-failure",
384                  )
385
386                  # 三个 started 事件保证：主失败发生前，两个清理任务都已在 await 中可被取消。
387                  # 这种事件握手比 sleep 后“猜任务大概启动了”更确定，不依赖机器速度。
388                  await primary_started.wait()
389                  await os_cleanup_started.wait()
390                  await value_cleanup_started.wait()
391                  release_primary.set()
392          # except* 会保留匹配部分的异常树形状，所以即使只有一个叶子，变量仍是异常组。
393          except* ToolBusinessError as business_group:
394              # 本例没有嵌套子组，所以 exceptions 中直接就是匹配叶子；生成器表达式惰性地产生
```

```
341  async def demonstrate_exception_groups() -> None:
392          # except* 会保留匹配部分的异常树形状，所以即使只有一个叶子，变量仍是异常组。
393          except* ToolBusinessError as business_group:
394              # 本例没有嵌套子组，所以 exceptions 中直接就是匹配叶子；生成器表达式惰性地产生
395              # 每条错误文字，extend 再逐项加入现有列表。
396              handled_business.extend(str(error) for error in business_group.exceptions)
397              print("  except* ToolBusinessError 处理:", handled_business)
398          except* OSError as os_group:
399              handled_os.extend(str(error) for error in os_group.exceptions)
400              print("  except* OSError 处理:", handled_os)
401      except ExceptionGroup as unhandled:
402          # 两个 except* 都未匹配 ValueError，因此它保持异常组结构来到外层普通 except。
403          remaining_group = unhandled
404          print(
405              "  外层 except 收到未处理子组:",
406              # 列表推导式遍历当前组的直接子异常，提取类型名供输出观察。
407              [type(error).__name__ for error in unhandled.exceptions],
408          )
409
410      # 断言既验证已处理叶子没有丢失，也验证未处理 ValueError 没有被前两个分支误吞。
411      assert handled_business == ["文档索引中没有该条目"]
412      assert handled_os == ["写入清理日志失败"]
413      assert remaining_group is not None
414      assert len(remaining_group.exceptions) == 1
415      assert isinstance(remaining_group.exceptions[0], ValueError)
416
417
418  async def as_best_effort_result(name: str, operation: Awaitable[str]) -> ToolResult:
419      """只把已知业务失败变成数据，取消和未知缺陷都自然传播。"""
420      # operation 是已经创建的可等待对象；await 成功后得到 str，再包装为成功 ToolResult。
421      # 包装发生在每个子 Task 内部，所以 TaskGroup 看到“可恢复业务失败”时，成员仍是正常返回。
422      try:
423          return ToolResult(name=name, value=await operation, error=None)
424      # 捕获范围刻意只包含已定义恢复语义的 ToolBusinessError。CancelledError 直接继承
425      # BaseException 而不是 Exception，ToolProgramError 也不是该业务类型，两者都会自然传播。
426      except ToolBusinessError as error:
```

```python
async def as_best_effort_result(name: str, operation: Awaitable[str]) -> ToolResult:
    """只把已知业务失败变成数据，取消和未知缺陷都自然传播。"""
    # operation 是已经创建的可等待对象；await 成功后得到 str，再包装为成功 ToolResult。
    # 包装发生在每个子 Task 内部，所以 TaskGroup 看到"可恢复业务失败"时，成员仍是正常返回。
    try:
        return ToolResult(name=name, value=await operation, error=None)
    # 捕获范围刻意只包含已定义恢复语义的 ToolBusinessError。CancelledError 直接继承
    # BaseException，而不是 Exception，ToolProgramError 也不是该业务类型，两者都会自然传播。
    except ToolBusinessError as error:
        # str(error) 保存适合展示的业务消息；value=None 明确表示没有工具值。
        return ToolResult(name=name, value=None, error=str(error))


async def demonstrate_best_effort() -> None:
    # 先验证"允许部分业务失败"：客户记录不存在被转成数据，另外两项仍成功。
    # 再注入程序缺陷，证明 best-effort 并不等于捕获所有异常。
    print_section("4. 相互独立：收集部分业务失败，程序缺陷仍失败")
    events: list[str] = []
    search: ControlledTool, record_call: ControlledTool, status: Controlle… = make_three_tools(
        events,
        record_error=ToolBusinessError("客户记录不存在"),
    )

    async with asyncio.TaskGroup() as group:
        # 每个 Task 运行的最外层协程都是 as_best_effort_result；它在内部 await 真实工具调用，
        # 所以业务异常会在离开成员 Task 前被转换，TaskGroup 不会因此取消其他成员。
        search_task: Task[ToolResult] = group.create_task(
            as_best_effort_result(search.name, search_docs(search)),
            name="best-effort-search",
        )
        record_task: Task[ToolResult] = group.create_task(
            as_best_effort_result(record_call.name, lookup_record(record_call)),
            name="best-effort-record",
        )
        status_task: Task[ToolResult] = group.create_task(
            as_best_effort_result(status.name, fetch_status(status)),
```

```
async def demonstrate_best_effort() -> None:
        )
        status_task: Task[ToolResult] = group.create_task(
            as_best_effort_result(status.name, fetch_status(status)),
            name="best-effort-status",
        )
        await wait_until_all_started(search, record_call, status)
        # 三项全部放行：search/status 返回值，record_call 抛可恢复的业务异常。
        release_all(search, record_call, status)

    # 成功越过 TaskGroup 后，三个 Task 都以 ToolResult 正常完成，可以统一取结果。
    results: list[ToolResult] = [search_task.result(), record_task.result(), status_task.result()]
    print("  部分成功结果:", results)
    # 列表推导式逐个读取 @property ok，预期为“成功、业务失败、成功”。
    assert [result.ok for result in results] == [True, False, True]
    assert results[1].error == "客户记录不存在"

    # 第二轮复用相同结构，但把 record 错误换成 ToolProgramError。该类型不会被包装器捕获。
    bug_events: list[str] = []
    bug_search: ControlledTool, bug_record: ControlledTool, bug_status: Controlle… = make_three_tools(
        bug_events,
        record_error=ToolProgramError("响应结构违反协议"),
    )
    bug_count = 0
    try:
        async with asyncio.TaskGroup() as group:
            bug_search_task: Task[ToolResult] = group.create_task(
                as_best_effort_result(bug_search.name, search_docs(bug_search))
            )
            bug_record_task: Task[ToolResult] = group.create_task(
                as_best_effort_result(bug_record.name, lookup_record(bug_record))
            )
            bug_status_task: Task[ToolResult] = group.create_task(
                as_best_effort_result(bug_status.name, fetch_status(bug_status))
            )
            await wait_until_all_started(bug_search, bug_record, bug_status)
            # 只放行错误项，让另外两个确定保持未完成，便于验证 fail-fast 对兄弟的取消。
```

```
431  async def demonstrate_best_effort() -> None:
484              )
485              await wait_until_all_started(bug_search, bug_record, bug_status)
486              # 只放行错误项，让另外两个确定保持未完成，便于验证 fail-fast 对兄弟的取消。
487              bug_record.release.set()
488              # 父 Task 留在组内等待，直到成员失败触发 TaskGroup 的内部取消。
489              await asyncio.Event().wait()
490      except* ToolProgramError as error_group:
491          bug_count: int = len(error_group.exceptions)
492          record(bug_events, "父请求：未知程序缺陷继续传播")
493
494      # 错误任务保存 ToolProgramError，未放行的两项则以 cancelled 状态结束。
495      print("  未知缺陷数:", bug_count)
496      assert bug_count == 1
497      assert bug_search_task.cancelled()
498      assert isinstance(bug_record_task.exception(), ToolProgramError)
499      assert bug_status_task.cancelled()
500
501
502  async def self_cancelling_child(started: asyncio.Event, events: list[str]) -> None:
503      # 这个成员主动取消自己，用来区分“单个成员取消”和“父请求被外部取消”。
504      record(events, "自取消子任务：开始")
505      started.set()
506      # current_task() 返回当前事件循环正在执行此协程的 Task；理论上也可能是 None，
507      # 因此先用断言收窄状态，再调用 Task.cancel() 给自己登记取消请求。
508      current: Task[Any] | None = asyncio.current_task()
509      assert current is not None
510      current.cancel()
511      try:
512          # cancel() 不是同步抛异常；sleep(0) 明确让出一次执行权，取消才在这个 await 处注入。
513          await asyncio.sleep(0)
514      finally:
515          # 即使 CancelledError 未在本函数捕获，finally 仍先执行，然后取消继续离开成员 Task。
516          record(events, "自取消子任务：清理完成")
517
518
519  async def demonstrate_child_cancellation() -> None:
```

```python

async def demonstrate_child_cancellation() -> None:
    # TaskGroup 把成员的 CancelledError 视作取消状态，而不是触发整个组 fail-fast 的普通失败；
    # 因此另一个兄弟仍可被放行并正常返回。
    print_section("5A. 单个子任务取消：不等于组失败")
    events: list[str] = []
    child_started = asyncio.Event()
    sibling = ControlledTool(
        "仍可完成的兄弟",
        "兄弟结果",
        events,
        asyncio.Event(),
        asyncio.Event(),
    )

    async with asyncio.TaskGroup() as group:
        cancelled_task: Task[None] = group.create_task(self_cancelling_child(child_started, events))
        sibling_task: Task[str] = group.create_task(execute_controlled_tool(sibling))
        # 先等两项都启动，避免在自取消任务尚未执行时就提前解释结果。
        await child_started.wait()
        await sibling.started.wait()
        sibling.release.set()

    # 组能够正常退出，但 cancelled_task.result() 仍会重新抛 CancelledError，所以这里只检查
    # cancelled()；正常兄弟则可以安全调用 result()。
    print("  自取消状态:", cancelled_task.cancelled())
    print("  兄弟结果:", sibling_task.result())
    assert cancelled_task.cancelled()
    assert sibling_task.result() == "兄弟结果"


async def waiting_best_effort_request(
    search: ControlledTool,
    record_call: ControlledTool,
    status: ControlledTool,
) -> list[ToolResult]:
    """外部取消实验的公共步骤：正常情况下三个工具会一直等待执行。"""
```

```python
async def waiting_best_effort_request(
    search: ControlledTool,
    record_call: ControlledTool,
    status: ControlledTool,
) -> list[ToolResult]:
    """外部取消实验的父请求：正常情况下三个工具会一直等待放行。"""
    async with asyncio.TaskGroup() as group:
        # 列表保存三个 Task 句柄。它们都用了 best-effort 包装，但该包装只处理业务异常，
        # 不会把上游取消转换成 ToolResult。
        tasks: list[Task[ToolResult]] = [
            group.create_task(as_best_effort_result(search.name, search_docs(search))),
            group.create_task(as_best_effort_result(record_call.name, lookup_record(record_call))),
            group.create_task(as_best_effort_result(status.name, fetch_status(status))),
        ]
        await wait_until_all_started(search, record_call, status)
        # 正常路径会一直停在这里；外部稍后取消承载本函数的父 Task。
        await asyncio.Event().wait()

    # 列表推导式只有在 TaskGroup 正常退出时才逐个取值。外部取消路径绝不应抵达这里；
    # 若到达，才说明三个工具都已有完整结果。
    return [task.result() for task in tasks]


async def demonstrate_external_cancellation() -> None:
    # 本实验从“请求调用方”视角创建并取消父 Task。取消父请求会进入其 TaskGroup，
    # 组再取消所有未完成成员、等待 finally，之后才把原 CancelledError 交还调用方。
    print_section("5B. 外部取消父请求：清理后仍向调用方传播取消")
    events: list[str] = []
    search: ControlledTool, record_call: ControlledTool, status: Controlle… = make_three_tools(events)
    # 这里故意用裸 asyncio.create_task，因为 parent_task 的所有者就是当前调用方；调用方会
    # 保存句柄并在下方 await 回收它。eager_start=False 同样把首次执行留给后续事件循环调度。
    parent_task: Task[list[ToolResult]] = asyncio.create_task(
        waiting_best_effort_request(search, record_call, status),
        name="agent-request",
        eager_start=False,
    )
```

```
572 async def demonstrate_external_cancellation() -> None:
586     # 先确认三个孩子都在工作，再取消父请求，排除“孩子其实从未启动”的歧义。
587     await wait_until_all_started(search, record_call, status)
588     # cancel() 只发取消请求；真正的清理与状态转换发生在 await parent_task 推进期间。
589     parent_task.cancel()
590     try:
591         await parent_task
592     except asyncio.CancelledError:
593         # 到达这里时，父 Task 内部的 TaskGroup 已等待三个工具完成 finally。
594         record(events, "调用方：仍收到 CancelledError")
595     else:
596         # 如果包装器错误地吞掉取消，await 会正常返回并走入这个显式失败分支。
597         raise AssertionError("best-effort 包装不应把外部取消变成成功结果")
598
599     # cancelled() 验证父请求最终保持取消语义；顺序断言验证清理早于调用方收到取消。
600     assert parent_task.cancelled()
601     assert_cleaned_before(
602         events,
603         "调用方：仍收到 CancelledError",
604         search.name,
605         record_call.name,
606         status.name,
607     )
608
609
610 async def owned_worker(
611     name: str,
612     started: asyncio.Event,
613     release: asyncio.Event,
614     events: list[str],
615 ) -> None:
616     # 这个最小 worker 只关心生命周期：started 表示已经归属并启动，release 表示允许完成，
617     # events 则留下取消和清理证据。返回 None 是因为实验观察的是状态而不是业务结果。
618     record(events, f"{name}：开始")
619     started.set()
620     try:
```

```python
async def owned_worker(
    try:
        await release.wait()
    except asyncio.CancelledError:
        # 与工具执行器一样，记录后重新抛出，维持 Task 的 cancelled 状态。
        record(events, f"{name}: 收到取消")
        raise
    finally:
        record(events, f"{name}: 清理完成")


async def wrongly_spawn_detached_child(
    started: asyncio.Event,
    release: asyncio.Event,
    events: list[str],
) -> asyncio.Task[None]:
    """教学反例：裸 create_task 创建的任务不属于调用方现有的 TaskGroup。"""
    # asyncio.create_task 只把任务登记到当前事件循环，不会扫描调用栈并自动加入调用方的
    # TaskGroup。即使本函数自身由某个 group 管理，detached 仍是另一个未被该组拥有的 Task。
    detached: Task[None] = asyncio.create_task(
        owned_worker("游离子任务", started, release, events),
        name="detached-child",
        eager_start=False,
    )
    # 等孩子真正开始后才返回句柄，确保外层演示拿到的不是一个尚未运行的模糊状态。
    await started.wait()
    return detached


async def spawn_child_in_given_group(
    group: asyncio.TaskGroup,
    started: asyncio.Event,
    release: asyncio.Event,
    events: list[str],
) -> asyncio.Task[None]:
    """显式接收所有者，把嵌套创建的任务登记进同一个组。"""
    # group 作为显式参数传入，函数签名就能说明“新任务归调用方给定的作用域所有”。
```

```python

async def spawn_child_in_given_group(
    group: asyncio.TaskGroup,
    started: asyncio.Event,
    release: asyncio.Event,
    events: list[str],
) -> asyncio.Task[None]:
    """显式接收所有者，把嵌套创建的任务登记进同一个组。"""
    # group 作为显式参数传入，函数签名就能说明“新任务归调用方给定的作用域所有”。
    # 返回 Task 句柄便于调用方稍后观察，但生命周期等待由 group 负责。
    return group.create_task(
        owned_worker("受组管理的子任务", started, release, events),
        name="owned-child",
    )


async def demonstrate_nested_ownership() -> None:
    # 先演示错误所有权：组内 launcher 裸创建的孩子会越过外层组。
    # 再演示正确写法：把现有 group 传入下层，由 group.create_task 登记孩子。
    print_section("6. 嵌套所有权：裸 create_task 不会自动入组")
    wrong_events: list[str] = []
    detached_started = asyncio.Event()
    detached_release = asyncio.Event()
    # 先用 None 表示句柄尚未取得；finally 据此判断是否有任务需要隔离回收。
    detached: asyncio.Task[None] | None = None

    try:
        async with asyncio.TaskGroup() as group:
            # group 只直接拥有 launcher。launcher 返回一个 Task 对象，并不会把这个返回值代表的
            # detached Task 自动登记为成员，这与 Java 方法返回 Future 后谁负责 join 类似。
            launcher: Task[Task[None]] = group.create_task(
                wrongly_spawn_detached_child(
                    detached_started,
                    detached_release,
                    wrong_events,
                )
            )
```

```
663  async def demonstrate_nested_ownership() -> None:
685          # 外层组只等 launcher 返回，因此现在可从 launcher.result() 取得游离 Task 句柄；
686          # detached 自己仍停在 release.wait()，这正是生命周期漏洞。
687          detached = launcher.result()
688          record(wrong_events, "外层 TaskGroup: 已经退出")
689          print("  反例中组退出时，游离任务仍未结束:", not detached.done())
690          assert not detached.done()
691      finally:
692          # 反例必须隔离：即使前面的断言失败，也显式取消并 await 游离任务。只调用 cancel()
693          # 而不 await 仍不足以回收，因为任务的 finally 需要事件循环继续推进。
694          if detached is not None and not detached.done():
695              detached.cancel()
696              try:
697                  await detached
698              except asyncio.CancelledError:
699                  # 这是演示隔离层主动发起且已经等待完成的取消，所以在本边界消费它。
700                  record(wrong_events, "隔离层：游离任务已回收")
701
702      # 时间线证明外层组退出发生在游离孩子清理之前，也就是它确实越过了错误的父作用域。
703      assert wrong_events.index("外层 TaskGroup: 已经退出") < wrong_events.index(
704          "游离子任务：清理完成"
705      )
706
707      # 正确轮使用全新的事件与时间线，避免前一轮已 set 的状态影响结论。
708      correct_events: list[str] = []
709      owned_started = asyncio.Event()
710      owned_release = asyncio.Event()
711      async with asyncio.TaskGroup() as group:
712          # launcher 本身和它通过同一个 group 新增的 owned-child 都是成员。TaskGroup 在退出期间
713          # 仍会等待后加入的成员，直到组内不再有未完成任务。
714          launcher: Task[Task[None]] = group.create_task(
715              spawn_child_in_given_group(group, owned_started, owned_release, correct_events)
716          )
717          await owned_started.wait()
718          owned_release.set()
719
```

```python
async def demonstrate_nested_ownership() -> None:

    # launcher 的返回值仍是子 Task 句柄；区别在于该 Task 已登记到 group，所以成功退出组时
    # 它必定完成。下面同时检查 done() 和清理标记的先后。
    owned_task: Task[None] = launcher.result()
    record(correct_events, "正确示例：外层 TaskGroup 已退出")
    print("  正确示例中组退出时，嵌套子任务已结束:", owned_task.done())
    assert owned_task.done()
    assert correct_events.index("受组管理的子任务：清理完成") < correct_events.index(
        "正确示例：外层 TaskGroup 已退出"
    )


async def demonstrate_whole_request_timeout() -> None:
    # 共同时间预算应包住整个 TaskGroup。期限触发时 timeout 先取消当前父 Task，父 Task 退出
    # TaskGroup 又会取消孩子并等待清理，最后 timeout 才把自己发起的取消转换为 TimeoutError。
    print_section("7. 整体 timeout 包围 TaskGroup：先回收，再抛 TimeoutError")
    events: list[str] = []
    search: ControlledTool, record_call: ControlledTool, status: Controlle… = make_three_tools(events)
    # 使用事件循环自己的单调时钟来设置期限，避免系统墙钟调整影响超时判断。
    loop: AbstractEventLoop = asyncio.get_running_loop()

    try:
        # 先不设期限；三个工具都开始后再把截止点设为“现在”。
        # 这仍是真实 timeout 取消路径，但不依赖机器速度或固定 sleep 时长。
        # asyncio.timeout(None) 先创建没有截止时间的异步上下文；as 绑定的对象可稍后 reschedule。
        async with asyncio.timeout(None) as timeout_scope:
            async with asyncio.TaskGroup() as group:
                # 三个 Task 都不保存句柄，因为本实验只验证统一取消和清理时间线。
                group.create_task(search_docs(search))
                group.create_task(lookup_record(record_call))
                group.create_task(fetch_status(status))
                await wait_until_all_started(search, record_call, status)
                # loop.time() 是“现在”；reschedule 后事件循环会尽快取消当前父 Task。
                timeout_scope.reschedule(loop.time())
                # 留在可取消点，让刚设定的期限有机会生效并进入 TaskGroup 的退出协议。
                await asyncio.Event().wait()
```

```python
async def demonstrate_whole_request_timeout() -> None:
    # TimeoutError 的转换发生在离开 timeout 上下文时，所以必须在上下文外捕获。
    except TimeoutError:
        record(events, "父请求：收到 TimeoutError")

    # 只有三个工具的 finally 都先完成，这个超时边界断言才会通过。超时不是硬杀任务。
    assert_cleaned_before(
        events,
        "父请求：收到 TimeoutError",
        search.name,
        record_call.name,
        status.name,
    )


async def demonstrate_gather_difference() -> None:
    # gather 也能并发等待多个 awaitable，但默认情况下一个成员抛普通异常时，它会把首个异常
    # 立即交给等待者，不会像 TaskGroup 那样自动取消并等待未完成兄弟。
    print_section("8. gather 对照：首个异常不会自动取消兄弟")
    events: list[str] = []
    failing = ControlledTool(
        "gather-失败项",
        "不会返回",
        events,
        asyncio.Event(),
        asyncio.Event(),
        ToolProgramError("gather 中的工具失败"),
    )
    sibling = ControlledTool(
        "gather-兄弟项",
        "兄弟继续完成",
        events,
        asyncio.Event(),
        asyncio.Event(),
    )
    # 这里显式保存两个裸 Task 的句柄，因为 gather 异常返回后，所有权和回收责任仍在调用方。
```

```
769  async def demonstrate_gather_difference() -> None:
789      # 这里显式保存两个裸 Task 的句柄，因为 gather 异常返回后，所有权和回收责任仍在调用方。
790      failing_task: Task[str] = asyncio.create_task(execute_controlled_tool(failing))
791      sibling_task: Task[str] = asyncio.create_task(execute_controlled_tool(sibling))
792
793      try:
794          # 先确保两项都已启动，再只放行失败项；兄弟仍确定停在自己的 release.wait()。
795          await wait_until_all_started(failing, sibling)
796          failing.release.set()
797          try:
798              # 默认 return_exceptions=False；failing_task 的 ToolProgramError 直接从 await 抛出。
799              await asyncio.gather(failing_task, sibling_task)
800          except ToolProgramError as error:
801              print("  gather 立即传播:", error)
802
803          # 捕获 gather 抛出的异常后，sibling_task 仍由事件循环运行，未被自动取消。
804          print("  捕获异常后，兄弟仍在运行:", not sibling_task.done())
805          assert not sibling_task.done()
806          # 调用方仍持有句柄，因此可以放行并 await，明确完成剩余任务的回收。
807          sibling.release.set()
808          print("  调用方主动回收兄弟结果:", await sibling_task)
809      finally:
810          # 若教学断言意外失败，也不能把 sibling 留给 asyncio.run 收尾。finally 覆盖正常、
811          # 异常和断言失败路径；只有 Task 尚未完成时才发送取消并等待它真正结束。
812          if not sibling_task.done():
813              sibling_task.cancel()
814              try:
815                  await sibling_task
816              except asyncio.CancelledError:
817                  # 清理兜底路径预期由本函数发起取消，此处消费它后即可继续原来的退出过程。
818                  pass
819
820      # 失败 Task 已结束但不是 cancelled；exception() 应返回最初的 ToolProgramError。
821      assert failing_task.done()
822      assert isinstance(failing_task.exception(), ToolProgramError)
823
824
```

```
823
824
825 async def run_examples() -> None:
826     # 每个 await 都等当前实验彻底结束后才进入下一项，避免各实验的事件与任务交错。
827     # 这份顺序只用于教学展示；每个实验内部的 Task 仍按各自并发规则调度。
828     await demonstrate_normal_task_group()
829     await demonstrate_all_required_failure()
830     await demonstrate_exception_groups()
831     await demonstrate_best_effort()
832     await demonstrate_child_cancellation()
833     await demonstrate_external_cancellation()
834     await demonstrate_nested_ownership()
835     await demonstrate_whole_request_timeout()
836     await demonstrate_gather_difference()
837     print("\n所有第 22 章示例与运行时断言均已完成。")
838
839
840 def main() -> None:
841     """同步入口只创建一次事件循环；所有并发任务都在受控作用域内回收。"""
842     # 调用 run_examples() 先产生顶层协程对象；asyncio.run 创建事件循环、运行它到结束，
843     # 最后关闭循环。它只能在当前线程没有正在运行的事件循环时调用。
844     asyncio.run(run_examples())
845
846
847 # 模块被直接作为脚本执行时 __name__ 等于 "__main__"，才调用同步入口；若被其他模块 import，
848 # 上面的类与函数只会完成定义，不会自动运行九个实验。这与 Java 的固定 main 方法查找不同。
849 if __name__ == "__main__":
850     main()
851
```